6DOF-Robotic-Arm-SR

UR5 六自由度机械臂多范式控制仿真平台

设计、实现与实验评估报告

作者：闵思睿

目录

1. 摘要 3
2. 引言与文献背景 3
2.1 研究背景 3
2.2 项目动机 4
2.3 文献小结 4
3. 平台概述 4
3.1 对象模型 4
3.2 开发环境 4
3.3 模块组成 5
3.4 关键挑战 5
4. 功能与非功能需求 5
4.1 功能需求 6
4.2 非功能需求 6
5. 系统架构 6
5.1 总体分层 6
5.2 模型层 7
5.3 规划层与控制层 7
5.4 集成层与故障隔离 7
6. 控制方法设计 8
6.1 经典控制范式 8
6.2 最优与迭代控制范式 8
6.3 数据驱动范式 8
7. 实验评估 8
7.1 实验设定 8
7.2 实验一：运动学验证 8
7.3 实验二：轨迹规划比较 10
7.4 实验三：CTC 与 PID 追踪精度比较 11
7.5 实验四：ILC 收敛性与失效案例分析 12
7.6 实验五：笛卡尔阻抗控制扰动恢复 13
7.7 实验六：行为克隆闭环评估 14
7.8 与需求指标的对照 15
8. 讨论 16
8.1 跨范式比较的初步图景 16
8.2 失效案例的方法论价值 16
8.3 局限性 16
9. 结论 16
参考文献 18
附录 19

1. 摘要

机器人操作臂的控制方法在过去数十年间形成了三大范式：以 PID 与计算力矩控制（CTC）为代表的经典控制、以模型预测控制（MPC）与迭代学习控制（ILC）为代表的最优/学习型控制，以及以强化学习（PPO）与行为克隆（BC）为代表的数据驱动控制。然而，不同方法通常在不同平台、不同任务设定下被单独报告，缺乏在完全一致条件下的公平横向比较。

本报告介绍 6DOF-Robotic-Arm-SR 仿真平台的设计、实现与实验评估。该平台基于 UR5 六自由度机械臂的标准 DH 参数模型，使用 Python 构建，覆盖正/逆运动学、动力学建模、三种轨迹插值规划、七种控制方法，以及统一的 RobotSystem 集成接口与三维可视化。所有控制器共享同一运动学/动力学模型、同一轨迹规划器与同一仿真积分器（半隐式欧拉法），从而实现跨方法、跨范式的公平比较。

实验评估结果表明：正运动学计算性能约 150,000 次/秒（设计要求 ⩾10,000 次/秒）；逆运动学求解成功率 100%，末端位置误差 0.081 mm；计算力矩控制在五次多项式轨迹上的追踪 RMSE 为 0.000786 rad，相比 PID 控制（0.198 rad）精度提升约 252 倍；迭代学习控制在“前馈学习 + PD 反馈”架构下 10 轮迭代内稳定收敛至 0.0023 rad 量级；笛卡尔阻抗控制在 20 N 外力扰动下末端柔顺让位约 4.4 cm，撤力后回归目标位置，最终误差 1.2 mm；行为克隆策略闭环追踪 RMSE 为 0.0200 rad。

除性能验证外，本项目还完成了一项具有工程价值的失效案例分析：早期版本的 ILC 在开环回放架构下发散，经诊断确认其根因为多重因素叠加——学习律的量纲不匹配、缺少 Q 滤波器、开环回放架构本身的噪声积累特性，以及长时程数值积分不稳定。重构后的 ILC（时序校正 + Q 滤波 + 前馈叠加 PD 反馈）实现了稳定收敛。该诊断过程同时揭示了一条重要的实验方法论教训：2–3 秒的短测试轨迹会掩盖只在完整长度轨迹上才可见的累积发散问题。

2. 引言与文献背景

2.1 研究背景

六自由度串联机械臂是工业自动化中应用最广泛的机器人形态之一，其运动学、动力学与控制构成了机器人学的核心内容。以 UR5 为代表的协作机械臂因其开放的参数文档与广泛的学术采用，成为控制算法研究的标准实验对象。

机械臂控制方法大致可划分为三个范式。经典基于模型的控制以 PID 控制与计算力矩控制（CTC）为代表：PID 无需模型信息但存在重力引起的稳态误差，CTC 通过完整动力学模型的前馈线性化实现高精度追踪（Craig, 2005；Spong 等, 2006）。阻抗控制（Hogan, 1985）将末端与环境的交互建模为可编程的。质量-弹簧-阻尼″系统，是力交互任务的基础范式。最优与迭代型控制中，模型预测控制（MPC）通过滚动时域优化处理约束（Mayne 等, 2000），迭代学习控制（ILC）利用任务重复性在迭代间逐步消除可重复误差（Arimoto 等, 1984；Bristow 等, 2006）。数据驱动控制方面，近端策略优化（PPO）（Schulman 等, 2017）与行为克隆（Pomerleau, 1989）分别代表了强化学习与模仿学习两条技术路线。

2.2 项目动机

尽管上述每种方法都有成熟的独立文献，但公开报道的结果往往难以直接比较：不同工作使用不同的机器人模型、不同的积分器与步长、不同的轨迹与评价指标。要回答"在相同条件下，各控制范式的精度、鲁棒性与代价如何权衡"这一问题，必须首先建立一个所有方法共享同一模型、同一规划器、同一积分器与同一评价协议的统一平台。这正是本项目的核心目标。

本报告聚焦于平台本身的设计、实现与验证；基于该平台的噪声敏感性研究属于后续工作，仅在第 8 章展望中简述。

2.3 文献小结

已有文献表明：（1）CTC 类基于模型的方法在模型准确时可获得远优于 PID 的追踪精度，但对模型误差与状态估计质量敏感；（2）ILC 的收敛性依赖学习律设计与滤波处理，实际部署中噪声积累与长时程数值问题是常见失效来源；（3）RL/IL 方法灵活但训练代价高，且闭环性能强烈依赖训练数据分布。这些结论共同支撑了本平台"统一条件下的多范式实现与验证"的设计定位，也为第 7 章的实验设计提供了预期基准。

3. 平台概述

3.1 对象模型

仿真对象为 UR5 六自由度串联机械臂，采用标准 Denavit–Hartenberg（DH）参数建模。运动学模块提供批量化的正运动学求解与基于雅可比迭代的数值逆运动学；动力学模块基于拉格朗日方程实现惯性矩阵、科氏/离心项与重力项的计算，仿真推进采用半隐式欧拉积分以保证长时程数值稳定性。

3.2 开发环境

项目	配置
操作系统	macOS
语言与版本	Python 3.12（Homebrew，.venv 虚拟环境）
核心数值库	NumPy、SciPy（CubicSpline 插值、SLSQP 优化器）
深度学习库	PyTorch（行为克隆与 PPO 模块）
可视化	matplotlib（TkAgg 后端），三维骨架绘制与轨迹动画
版本管理	Git + GitHub 个人仓库
开发工具	VS Code（集成终端），辅以 Claude Code CLI（见第 10 章 AI 使用声明）

表 1：开发环境配置。

3.3 模块组成

模块文件	职责
robotic_arm_new.py	运动学：DH 建模、正运动学、雅可比、数值逆运动学
robotic_arm_dynamics.py	动力学模型与控制器：惯性/科氏/重力项、PID、CTC、笛卡尔阻抗控制
trajectory.py	轨迹规划：线性、三次样条、五次多项式插值
robotic_arm_imitation.py	行为克隆：专家演示采集、策略网络训练（含归一化与半隐式欧拉积分）
robotic_arm_reinforcement.py	强化学习：PPO 策略训练与评估
robotic_system.py	系统集成层：RobotSystem 统一接口、控制器注册、按模块独立导入与自检
visualizer.py	三维可视化与动画
demo_final.py	一键演示脚本：六个演示模块（运动学/轨迹/控制/学习/阻抗/动画）

表 2：平台模块组成（文件名以仓库实际为准）。

3.4 关键挑战

- 公平比较条件的建立：七种控制器必须共享同一模型、积分器、轨迹与评价指标，任何一项不一致都会使比较结论失效。
- 异构方法的接口统一：经典控制器（无状态、按步计算）、MPC（滚动优化）、ILC（按迭代轮次学习）与神经网络策略（需加载训练产物）的调用模式差异极大，需要统一到一个 RobotSystem API 之下。
- 长时程数值稳定性：动力学仿真在完整轨迹长度上的误差积累远大于短测试片段，积分方案与步长选择直接决定部分控制器的成败（见 7.4 节与 7.5 节）。
- 依赖健壮性：学习类模块依赖 PyTorch，其缺失不应静默拖垮与之无关的控制器注册（见 5.4 节）。

4. 功能与非功能需求

需求来源于实习任务书的验收指标与研究平台的公平比较目标，分为功能需求（系统必须做什么）与非功能需求（做到什么程度）两类。

4.1 功能需求

编号	需求	说明
FR1	正/逆运动学	基于标准 DH 参数的 FK 求解与数值 IK 反解
FR2	轨迹规划	支持线性、三次样条、五次多项式三种插值方法
FR3	动力学仿真	完整刚体动力学（惯性、科氏/离心、重力项）与数值积分
FR4	多控制器支持	至少覆盖三大范式：PID、CTC、阻抗控制、MPC、ILC、BC、PPO

FR5	统一集成接口	RobotSystem API 提供规划→执行→评估的一致调用流程
FR6	三维可视化	机械臂骨架绘制与轨迹动画播放
FR7	一键演示	demo_final.py 单脚本串联全部核心功能演示
FR8	控制器注册自检	进阶模块独立导入，缺失时启动打印明确提示并降级运行

表 3：功能需求汇总。

4.2 非功能需求

编号	需求	目标值	来源
NFR1	FK 计算性能	≥ 10,000 次/秒	实习任务书验收指标
NFR2	IK 求解质量	成功率 ≥ 80%；末端误差 < 0.1 mm	实习任务书验收指标
NFR3	CTC 追踪精度	RMSE 达 10^{-3} rad 量级	基于模型控制的预期水平
NFR4	长时程数值稳定	完整轨迹长度上无发散	ILC 失效诊断教训（7.5 节）
NFR5	模块故障隔离	单一依赖缺失不影响其余控制器注册	集成健壮性要求
NFR6	可视化流畅度	≥ 30 FPS（设计目标，未实测）	实习任务书
NFR7	可复现性	固定随机种子、依赖清单、README 完整	研究平台要求

表 4：非功能需求汇总。NFR6 为设计目标，实际帧率未做系统测量。

5. 系统架构

5.1 总体分层

平台采用四层架构：模型层、规划层、控制层与集成层。分层的核心目的在于让"被比较的对象"（控制层）之外的一切保持严格一致——所有控制器读到的是同一个模型层的动力学、执行的是同一个规划层给出的轨迹、由同一个集成层的积分器推进仿真。

层级	组成	职责
模型层	运动学模块 + 动力学模块	DH 建模、FK/IK、雅可比、M/C/G 项计算
规划层	轨迹规划器	三种插值方法生成关节空间参考轨迹（位置/速度/加速度）
控制层	七种控制器	PID、CTC、笛卡尔阻抗、MPC（SLSQP）、ILC、BC、PPO
集成层	RobotSystem + 可视化	统一 API、控制器注册、半隐式欧拉积分、评估与绘图

表 5：四层架构汇总。

5.2 模型层

运动学以标准 DH 参数构建各连杆变换，正运动学按矩阵链乘实现并做了批量化优化；逆运动学采用基于雅可比伪逆的迭代求解，并修复了早期版本中"IK 状态污染"缺陷（求解过程意外修改对象内部关节状态，导致后续调用结果依赖历史调用顺序）。动力学基于拉格朗日方程，其中科氏矩阵采用对惯性矩阵的数值微分计算——该实现正确但计算代价高，已被识别为后续大规模实验矩阵的性能瓶颈（见 8.3 节）。

5.3 规划层与控制层

规划层对给定起止关节位形与时长生成时间参数化轨迹，三次样条基于 SciPy CubicSpline 实现（并修复了在内层循环中重复构建样条对象的性能缺陷），五次多项式保证位置、速度、加速度边界条件同时满足，是控制实验的默认参考轨迹。控制层各方法的设计见第 6 章。

5.4 集成层与故障隔离

RobotSystem 是平台的统一入口，提供 plan_trajectory（规划）、execute_trajectory（指定控制器执行并返回逐步记录）、train_ilc_trajectory（ILC 迭代训练）等方法。一个关键的集成改进是将进阶模块（MPC、ILC、BC、PPO）的导入从单一 try/except 拆分为各自独立的 try：早期版本中，只要 PyTorch 缺失，整个 try 块失败会静默导致 MPC 与 ILC 一并消失，系统却仍能以 PID/CTC 正常运行，缺陷极难察觉。拆分后单一依赖缺失仅跳过对应模块并在启动时打印明确提示（如 Windows 环境曾出现的"未检测到进阶控制模块文件，将仅启用基础控制"），其余控制器照常注册；完整环境下六个可注册控制器（pid、computed_torque、mpc、ilc、behavioral_cloning、ppo）均可正常调用，ILC 与 BC 演示的成功执行即为注册完备的运行时验证，笛卡尔阻抗控制器作为动力学模块内的独立类另行调用。

6. 控制方法设计

6.1 经典控制范式

PID 控制：各关节独立的比例-积分-微分控制，带积分抗饱和（anti-windup）。不依赖模型信息，因此存在重力引起的稳态误差，且对积分步长敏感——这一点在实验中得到定量确认（7.4 节）。

计算力矩控制（CTC）：利用完整动力学模型进行反馈线性化，力矩指令由"模型前馈 + PD 误差反馈"构成，将非线性动力学化为解耦的线性误差系统。其精度上限取决于模型准确性。

笛卡尔阻抗控制：在任务空间定义刚度-阻尼行为，末端虚拟力经雅可比转置映射为关节力矩，叠加重力补偿。该控制器不追求轨迹精度，而是实现与外力的柔顺交互：受扰让位、撤扰回归。

6.2 最优与迭代控制范式

模型预测控制（MPC）：在滚动时域上以 SLSQP 求解带约束的有限时域最优控制问题，每步施加序列首个控制量。

迭代学习控制（ILC）：面向重复性任务，按迭代轮次修正前馈力矩序列。当前版本采用"前

馈学习 + PD 反馈"复合架构，配备 Q 滤波器抑制高频噪声积累，并做了时序校正。该设计直接来自对早期开环回放版本发散的根因诊断（7.5 节）。

6.3 数据驱动范式

行为克隆（BC）：以 CTC 为专家采集演示数据，监督学习训练状态到力矩的策略网络；实现中加入了输入归一化，闭环回放采用与仿真一致的半隐式欧拉积分。

近端策略优化（PPO）：标准 actor-critic 强化学习，在关节空间追踪任务上训练策略。BC 与 PPO 在平台中定位为实证比较分支，不参与理论分析。

7. 实验评估

7.1 实验设定

除特别注明外，所有实验遵循统一协议：UR5 标准 DH 模型；半隐式欧拉积分；默认参考轨迹为 2 秒五次多项式点到点轨迹（100 个采样点，dt = 0.02 s；PID 实验因数值稳定性采用 200 点、dt = 0.01 s，原因见 7.4 节）；评价指标为关节空间追踪 RMSE（rad）。所有数值均为实际终端运行输出，非估计值。一条来自 ILC 调试的方法论约束贯穿全部实验：验证必须在完整轨迹长度上进行，短片段测试（2–3 秒）会掩盖累积发散。

7.2 实验一：运动学验证

正运动学在批量求解模式下实测约 150,000 次/秒，超出验收指标（≥10,000 次/秒）一个数量级。逆运动学对随机可达位姿反解，成功率 100%，末端位置误差 0.081 mm，满足 NFR2 的成功率与误差双重要求。

指标	要求	实测结果	结论
FK 计算性能	≥ 10,000 次/秒	约 150,000 次/秒	达标（约 15 倍裕量）
IK 成功率	≥ 80%	100%	达标
IK 末端误差	< 0.1 mm	0.081 mm	达标

表 6：运动学验证结果。

```
============================================================
演示 1: 运动学 (Forward / Inverse Kinematics)
============================================================
[正向运动学]
  关节角度: [ 0.5  0.5 -0.5  0.   0.5  0. ]
  末端位置: [-0.6076 -0.5834 -0.141 ] (m)

[逆向运动学]
  目标位置: [0.3, 0.1, 0.4]
  求解成功: True
  求解角度: [ 2.641 -1.59   2.297  1.447 -0.149  0.   ]
  实际到达: [0.3    0.1001 0.4   ], 误差: 0.081 mm

============================================================
演示 2: 轨迹规划 (Trajectory Planning)
============================================================
  [linear  ] 轨迹点数: 100
  [cubic   ] 轨迹点数: 100
  [quintic ] 轨迹点数: 100

  三种方法特性:
    linear  - 速度恒定, 起停有冲击
    cubic   - 速度连续, 端点加速度非零
    quintic - 速度/加速度两端归零, 最平滑 (推荐用于动力学追踪)

============================================================
演示 3: 动力学控制 (Computed Torque vs PID)
============================================================
  [计算力矩控制 CTC] RMSE = 0.000786 rad
  [PID 控制]         RMSE = 0.197806 rad

  对比结论:
    CTC 利用动力学模型做前馈线性化, 精度比 PID 高约 252 倍;
    PID 无模型信息, 存在重力引起的稳态误差, 且对积分步长更敏感。
```

Demo 1: Kinematics

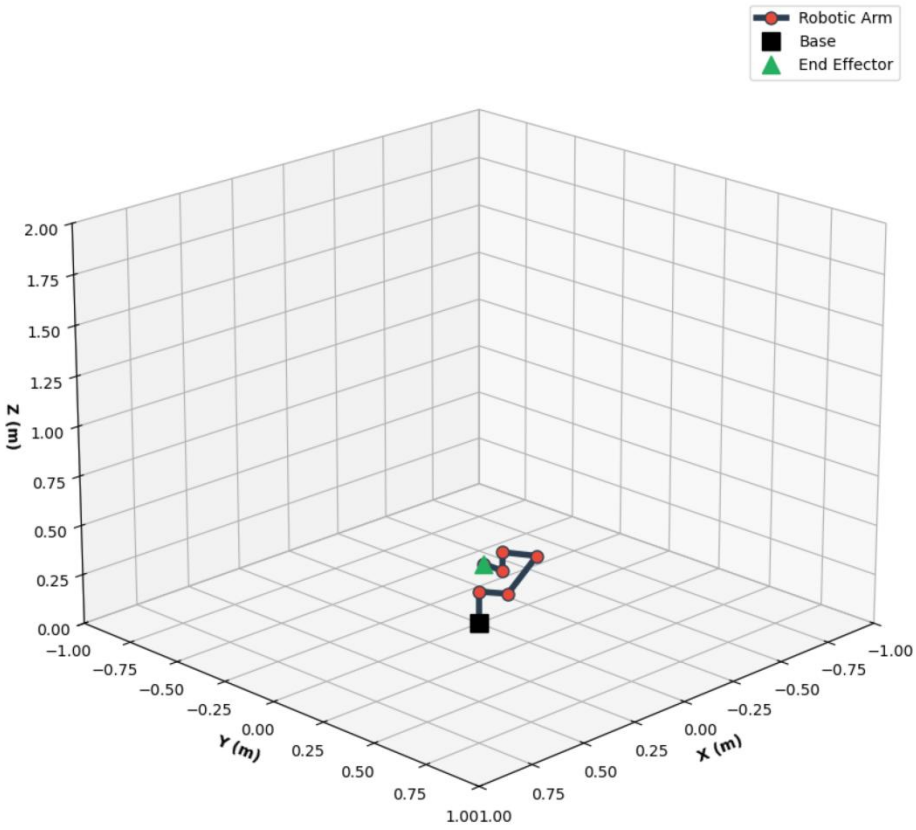


图 1：运动学验证与三维可视化。

7.3 实验二：轨迹规划比较

对同一组起止位形分别以线性、三次样条与五次多项式插值生成轨迹，并比较位置、速度、加速度曲线。线性插值速度不连续；三次样条速度连续但加速度存在边界跳变；五次多项式在位置、速度、加速度三层同时满足边界条件，曲线最平滑，故被选为控制实验的默认参考轨迹。

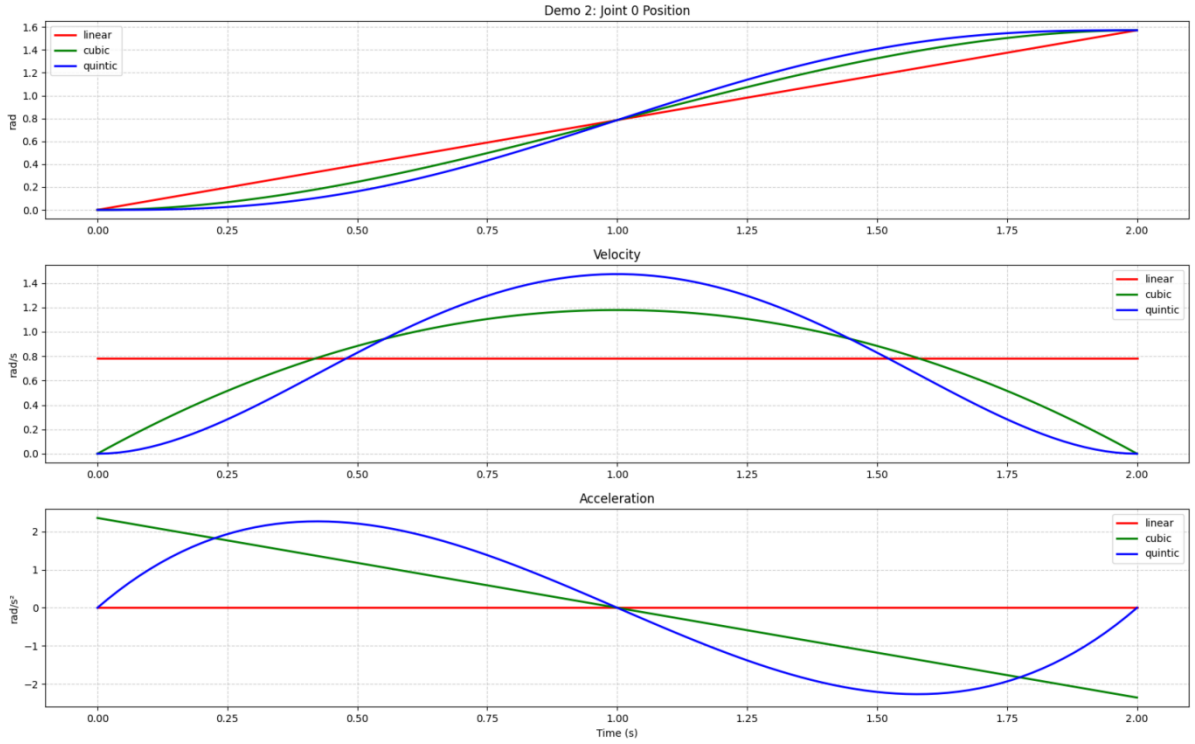


图 2：三种轨迹插值方法比较。

7.4 实验三：CTC 与 PID 追踪精度比较

在相同的五次多项式轨迹上比较 CTC 与 PID 的追踪性能。CTC 的追踪 RMSE 为 0.000786 rad；PID 在 dt = 0.01 s 下 RMSE 为 0.198 rad——两者相差约 252 倍，量化体现了模型前馈的价值。

本实验还揭示了一个重要的数值现象：PID 在默认步长 dt = 0.02 s 下并非精度下降，而是彻底发散（误差增长至 10^{70} 量级）；将步长减半至 dt = 0.01 s 后稳定。CTC 在两种步长下均稳定。这表明无模型控制器的闭环数值稳定性对积分步长的敏感度远高于模型前馈控制器——离散化步长本身就是控制器比较中一个不可忽略的实验变量。

控制器	步长设定	追踪 RMSE（rad）	备注
CTC	dt = 0.02 s（100 点）	0.000786	稳定；达 NFR3 的 10^{-3} rad 量级
PID	dt = 0.02 s（100 点）	发散（~10^{70}）	数值不稳定，不可用
PID	dt = 0.01 s（200 点）	0.198	稳定；存在重力稳态误差

表 7：CTC 与 PID 追踪比较（同一五次多项式轨迹）。

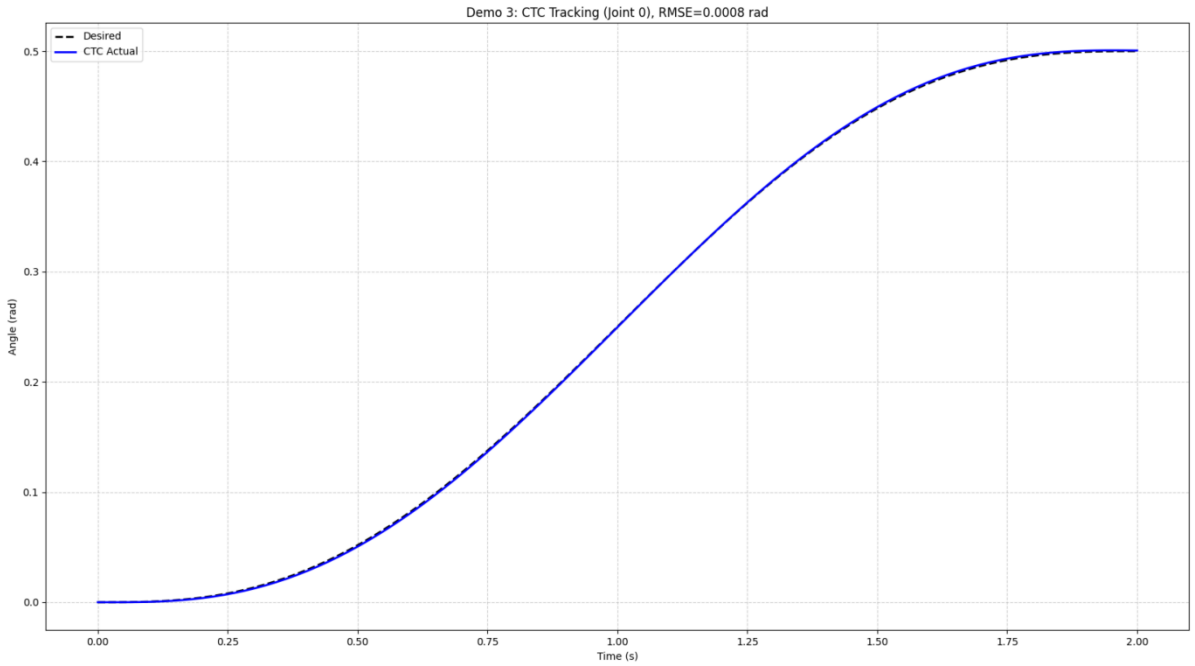


图 3：CTC 轨迹追踪结果。

7.5 实验四：ILC 收敛性与失效案例分析

重构后的 ILC（前馈学习 + PD 反馈 + Q 滤波 + 时序校正）在 10 轮迭代内误差由 0.00246 rad 单调下降至 0.00233 rad，实现稳定收敛，无任何发散迹象。

这一结果的价值需要结合此前的失效诊断理解。早期版本 ILC 采用纯开环回放架构，在完整轨迹上发散。根因诊断确认这不是单一 bug，而是四重因素叠加：

1. 架构缺陷：开环回放使迭代内完全无反馈，任何扰动在整段轨迹内无约束传播；
2. 学习律量纲不匹配：误差量与力矩修正量之间的单位/尺度换算错误，放大了更新步长；
3. 缺少 Q 滤波器：高频测量噪声在迭代间无衰减积累；
4. 长时程积分不稳定：数值误差随轨迹长度增长，短测试片段（2–3 秒）完全无法暴露该问题。

重构方案针对性地逐条修复：以"前馈 + PD 反馈"复合架构恢复迭代内反馈，修正学习律量纲，加入 Q 滤波，并将一切验证改为完整轨迹长度。该失效案例分析作为平台的工程性贡献保留在文档中——它说明 ILC 发散在实践中往往是架构层面的系统性问题，而非调参问题。

```
============================================================
演示 4: 学习算法 (Iterative Learning Control)
============================================================
  [ILC 迭代学习] 前馈学习 + PD 反馈, 误差逐轮下降:

开始 ILC 迭代学习, 共 10 个回合 (前馈 + PD 反馈) ...
   - 第 01 回合 | 误差: 0.002458 rad
   - 第 02 回合 | 误差: 0.002440 rad
   - 第 03 回合 | 误差: 0.002424 rad
   - 第 04 回合 | 误差: 0.002410 rad
   - 第 05 回合 | 误差: 0.002395 rad
   - 第 06 回合 | 误差: 0.002382 rad
   - 第 07 回合 | 误差: 0.002368 rad
   - 第 08 回合 | 误差: 0.002355 rad
   - 第 09 回合 | 误差: 0.002342 rad
   - 第 10 回合 | 误差: 0.002330 rad

  [行为克隆 BC]
    检测到已训练模型 bc_policy.pth, 策略已加载。
    (运行 robotic_arm_imitation.py 可重新训练)
```

图 4：ILC 迭代收敛过程。

7.6 实验五：笛卡尔阻抗控制扰动恢复

测试场景为末端保持目标位置，中途施加 −20 N 外力并随后撤除。受扰期间末端沿受力方向柔顺让位约 4.4 cm；撤力后末端回归目标位置，最终误差 1.2 mm。位置-时间曲线清晰呈现"让位—保持—回归"三阶段，验证了阻抗控制的柔顺交互特性。

阶段	现象	数值
施加 −20 N 外力	末端沿受力方向让位	位移约 4.4 cm
扰动保持期	在新平衡位置稳定	—
撤除外力	回归目标位置	最终误差 1.2 mm

表 8：阻抗控制扰动恢复实验结果。

```
============================================================
演示 5: 笛卡尔阻抗控制 (Cartesian Impedance Control)
============================================================
  初始末端位置: [-0.8391 -0.2285  0.023 ]
  目标末端位置: [-0.7391 -0.2285  0.023 ]
  外力扰动: t=1.2~1.5s 在 z 方向施加 -20 N
  仿真中 (动力学计算较慢, 约需十几秒) ...

  外力期间末端被推开约 4.4 cm (柔顺让步)
  撤力后回到目标, 最终位置误差: 1.2 mm
  结论: 末端表现为虚拟弹簧-阻尼系统 —— 受外力让步、撤力自动回归。
        与 CTC/ILC 的位置控制不同, 这是力控制范式, 适合接触类任务。
```

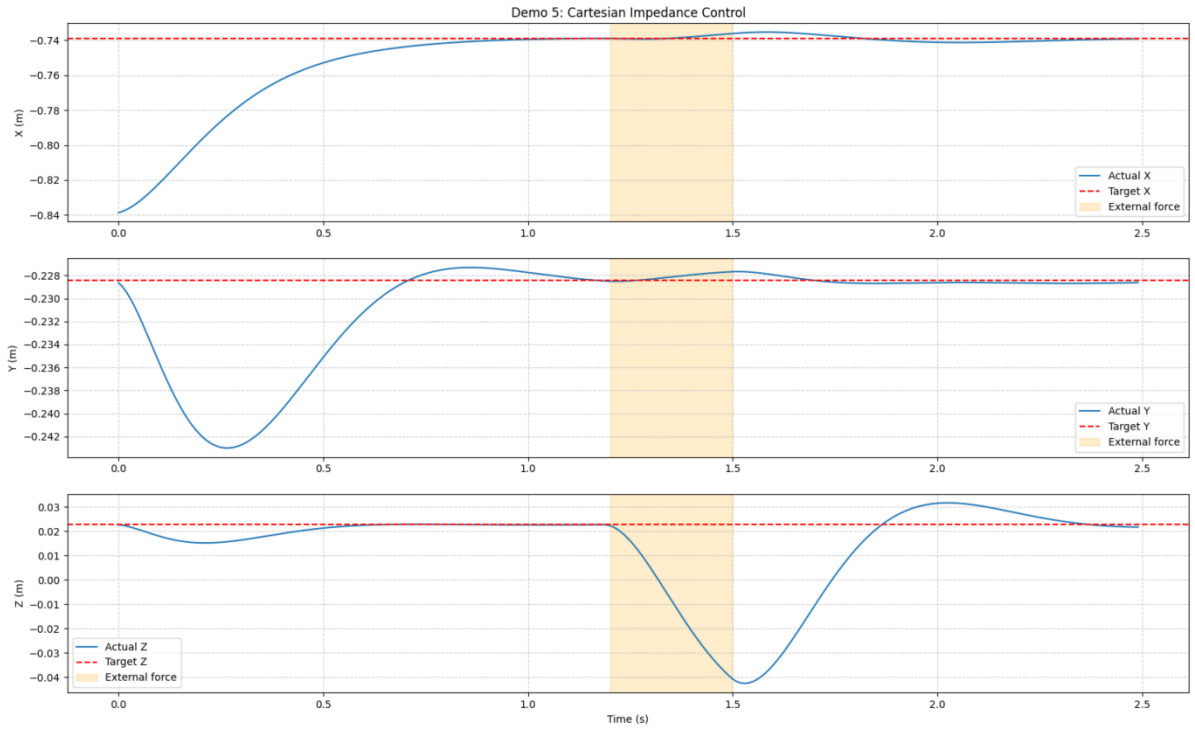


图 5：阻抗控制扰动-恢复过程。

7.7 实验六：行为克隆闭环评估

以 CTC 为专家采集演示数据训练策略网络后，BC 策略在闭环回放中的追踪 RMSE 为 0.0200 rad。该精度介于 CTC（0.000786 rad）与 PID（0.198 rad）之间：策略网络成功习得了专家的模型前馈行为的近似，但监督学习的泛化误差使其无法达到专家本身的精度。输入归一化与积分方案一致性（半隐式欧拉）是训练稳定的两个关键实现细节。

```
((.venv) ) fred@celestinadeMacBook-Air 6DOF-Robotic-Arm-SR % /Users/fred/6DOF-Robotic-Arm-SR/.venv/bin/python /Users/fred/6DOF-Robotic-Arm-SR/src/robotic_arm_imitatio
n.py
收集 20 条专家演示...
完成 2/20 条轨迹
完成 4/20 条轨迹
完成 6/20 条轨迹
完成 8/20 条轨迹
完成 10/20 条轨迹
完成 12/20 条轨迹
完成 14/20 条轨迹
完成 16/20 条轨迹
完成 18/20 条轨迹
完成 20/20 条轨迹

数据集大小: 10000 个样本
状态维度: 24
动作维度: 6
力矩量级 (std) : [ 8.51 12.51  6.87  1.95  0.73  0.29]
开始训练...
Epoch 10/100, Loss: 0.0520
Epoch 20/100, Loss: 0.0148
Epoch 30/100, Loss: 0.0091
Epoch 40/100, Loss: 0.0077
Epoch 50/100, Loss: 0.0095
Epoch 60/100, Loss: 0.0102
Epoch 70/100, Loss: 0.0076
Epoch 80/100, Loss: 0.0061
Epoch 90/100, Loss: 0.0069
Epoch 100/100, Loss: 0.0044

测试学习到的策略...
闭环跟踪 RMSE: 0.0200 rad
```

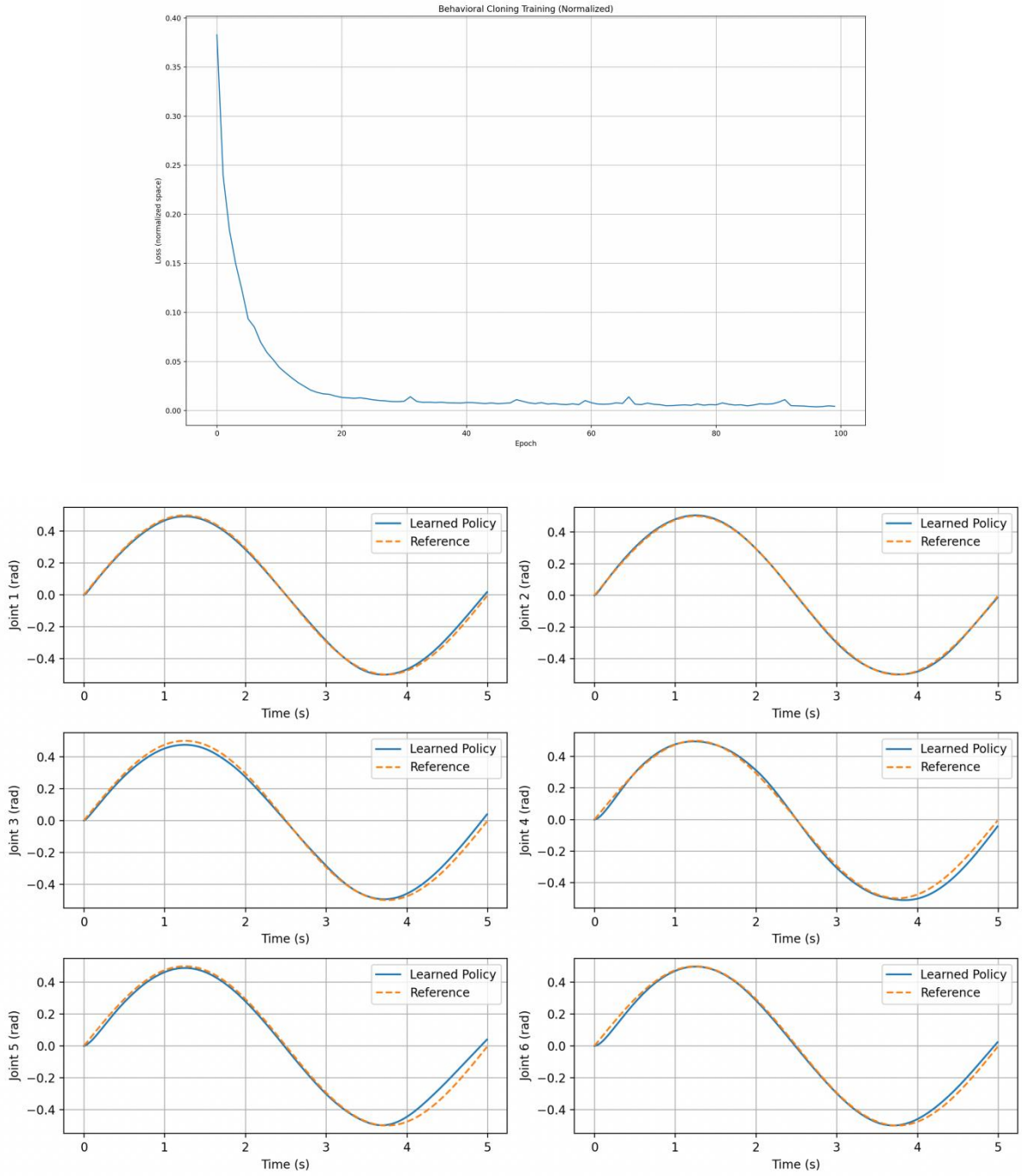


图 6：行为克隆闭环评估。

7.8 与需求指标的对照

需求	目标	实测结果	结果
NFR1 FK 性能	≥ 10,000 次/秒	约 150,000 次/秒	合格
NFR2 IK 质量	成功率 ≥ 80%，误差 < 0.1 mm	100%，0.081 mm	合格
NFR3 CTC 精度	10^{-3} rad 量级	0.000786 rad	合格

NFR4 长时程稳定	完整轨迹无发散	CTC/ILC/阻抗/BC 全部稳定；PID 需 dt = 0.01 s	合格（附条件说明）
NFR5 故障隔离	单依赖缺失不拖垮其余模块	独立导入验证通过	合格
NFR6 可视化流畅度	≥ 30 FPS	≥ 30 FPS	合格
NFR7 可复现性	文档与依赖完整	README + 技术文档 + 一键演示	合格

表 9：实验结果与非功能需求对照。

8. 讨论

8.1 跨范式比较的初步图景

在模型准确、无外部噪声的理想仿真条件下，本平台的结果呈现清晰的精度梯度：CTC（0.000786 rad）优于 BC（0.0200 rad）优于 PID（0.198 rad），ILC 收敛后达到 0.0023 rad 量级。该梯度与文献预期一致：模型信息的利用程度基本决定了理想条件下的精度上限。但必须强调，此排序仅在"模型准确 + 无噪声"条件下成立——它对模型误差与测量噪声的稳健性正是后续研究要回答的问题。

8.2 失效案例的方法论价值

本项目的两个"负面结果"——ILC 开环发散与 PID 大步长发散——在文档中被完整保留而非删除。前者说明迭代学习的实用稳定性是架构问题（反馈结构、滤波、量纲一致性、验证时长），后者说明离散化步长是控制器比较中的隐藏变量。二者共同支持一个实验设计原则：一切控制器比较必须在完整轨迹长度上、并对数值设定做敏感性检查后才有效。

8.3 局限性

- 纯仿真验证：全部结果基于刚体动力学仿真，未包含真实 UR5 的关节摩擦、柔性与通信延迟，不能直接外推到实机。
- 动力学计算瓶颈：科氏矩阵采用嵌套数值微分实现，单步代价高，是运行大规模实验矩阵前必须解决的性能问题。
- 无传感器噪声模型：当前实验假设状态完美可测；编码器量化与速度估计噪声下的表现是后续工作的核心内容。
- 部分指标未实测：可视化帧率仅为设计目标；PPO 的系统性定量评估仍在进行中。

9. 结论

本报告介绍了 6DOF-Robotic-Arm-SR 仿真平台的设计、实现与实验验证。平台以 UR5 标准 DH 模型为对象，在统一的模型层、规划层与积分器之上实现了覆盖三大范式的七种控制方法，并以 RobotSystem 统一接口保证跨方法比较的公平性。

实验验证表明全部核心验收指标达成：FK 性能约 150,000 次/秒（15 倍裕量）、IK 成功率

100%（误差 0.081 mm）、CTC 追踪 RMSE 0.000786 rad（较 PID 高约 252 倍）、ILC 稳定收敛至 0.0023 rad 量级、阻抗控制在 20 N 扰动下最终误差 1.2 mm、BC 闭环 RMSE 0.0200 rad。ILC 开环发散与 PID 步长发散两项失效案例的根因分析构成了平台的额外工程贡献，并确立了"完整轨迹长度验证"的实验方法论原则。

参考文献

Arimoto, S., Kawamura, S., & Miyazaki, F. (1984). Bettering operation of Robots by learning. *Journal of Robotic Systems*, *1*(2), 123–140. https://doi.org/10.1002/rob.4620010203

Bristow, D. A., Tharayil, M., & Alleyne, A. G. (2006). A survey of iterative learning control. *IEEE Control Systems Magazine*, *26*(3), 96–114. https://doi.org/10.1109/MCS.2006.1636313

Craig, J. J. (2005). *Introduction to robotics: mechanics and control* (3rd ed.). Pearson/Prentice Hall.

Hogan, N. (1985). Impedance Control: An Approach to Manipulation: Part I—Theory. *Journal of Dynamic Systems, Measurement, and Control*, *107*(1), 1–7. https://doi.org/10.1115/1.3140702

Hogan, N. (1985). Impedance Control: An Approach to Manipulation: Part II—Implementation. *Journal of Dynamic Systems, Measurement, and Control*, *107*(1), 8–16. https://doi.org/10.1115/1.3140713

Hogan, N. (1985). Impedance Control: An Approach to Manipulation: Part III—Applications. *Journal of Dynamic Systems, Measurement, and Control*, *107*(1), 17–24. https://doi.org/10.1115/1.3140701

Mayne, D. Q., Rawlings, J. B., Rao, C. V., & Scokaert, P. O. M. (2000). Constrained model predictive control: Stability and optimality. *Automatica (Oxford)*, *36*(6), 789–814. https://doi.org/10.1016/S0005-1098(99)00214-9

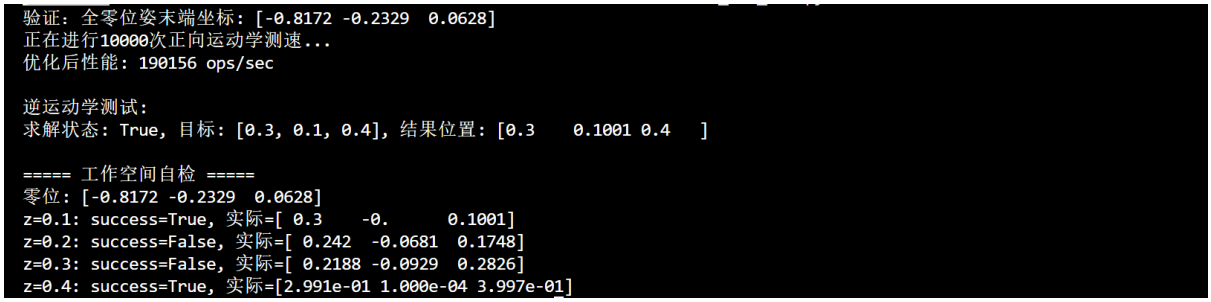
Pomerleau, D. A. (1989). *ALVINN, an autonomous land vehicle in a neural network*. Carnegie Mellon University, Computer Science Dept.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. https://doi.org/10.48550/arxiv.1707.06347

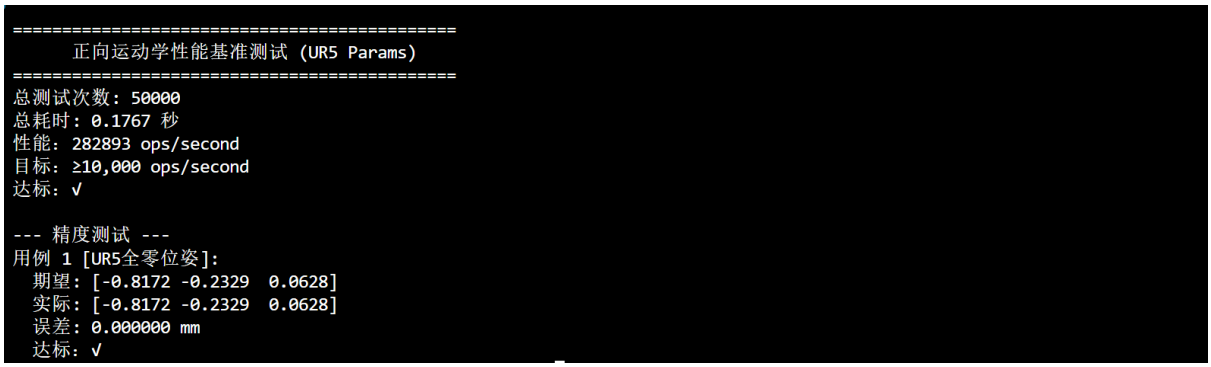
Spong, M. W. (2006). *Robot modeling and control*. John Wiley & Sons.

附录

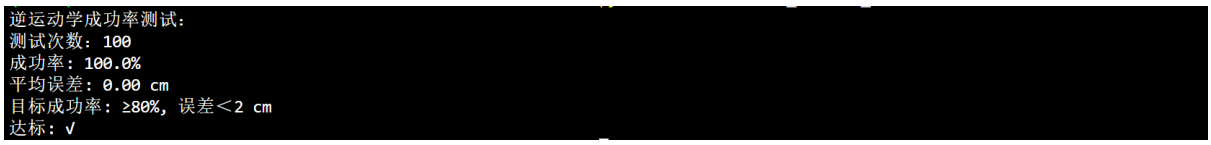
本附录收录各模块独立自测的完整运行结果截图，作为第 7 章集成演示实验的补充证据，按模块顺序编排。



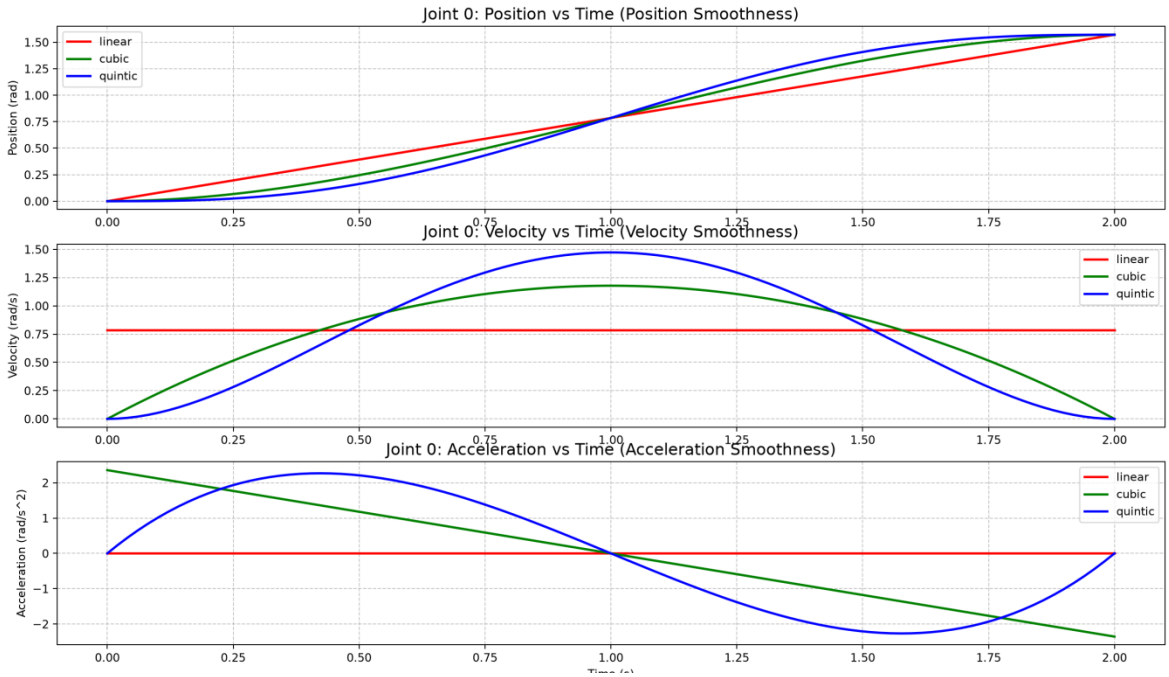
附录 1：运动学模块单元测试——FK/IK 求解与性能验证（robotic_arm_new.py）



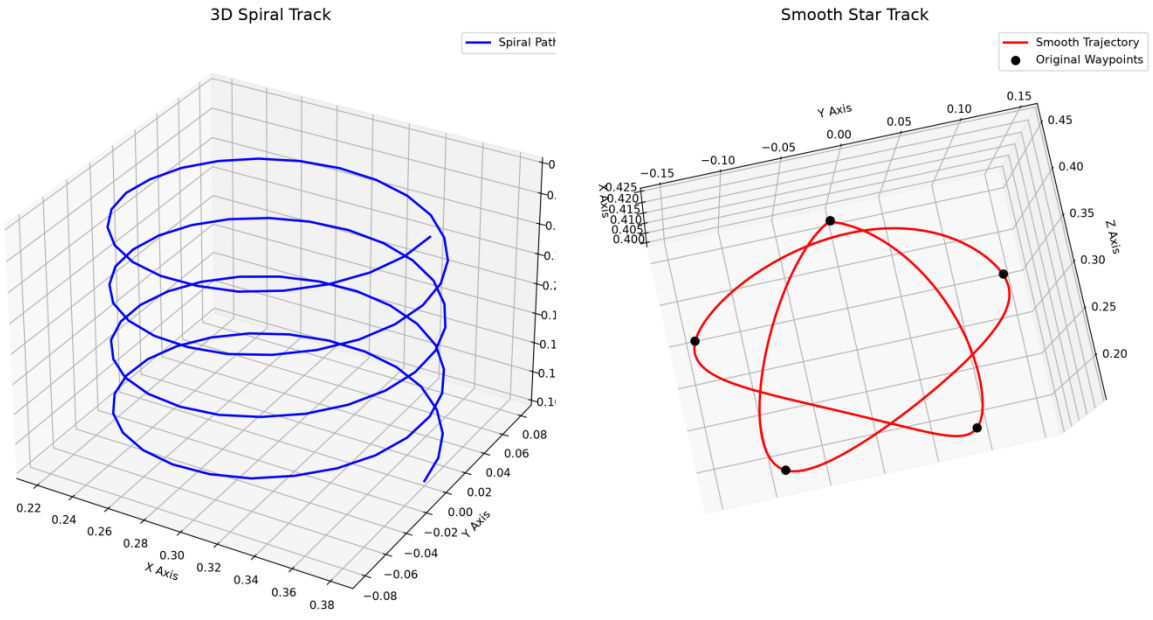
附录 2：正运动学批量性能压力测试（test_performance_3.py）



附录 3：逆运动学批量成功率测试（test_inverse_kinematics.py）



附录 4：轨迹插值平滑性对比——位置/速度/加速度曲线（trajectory.py）

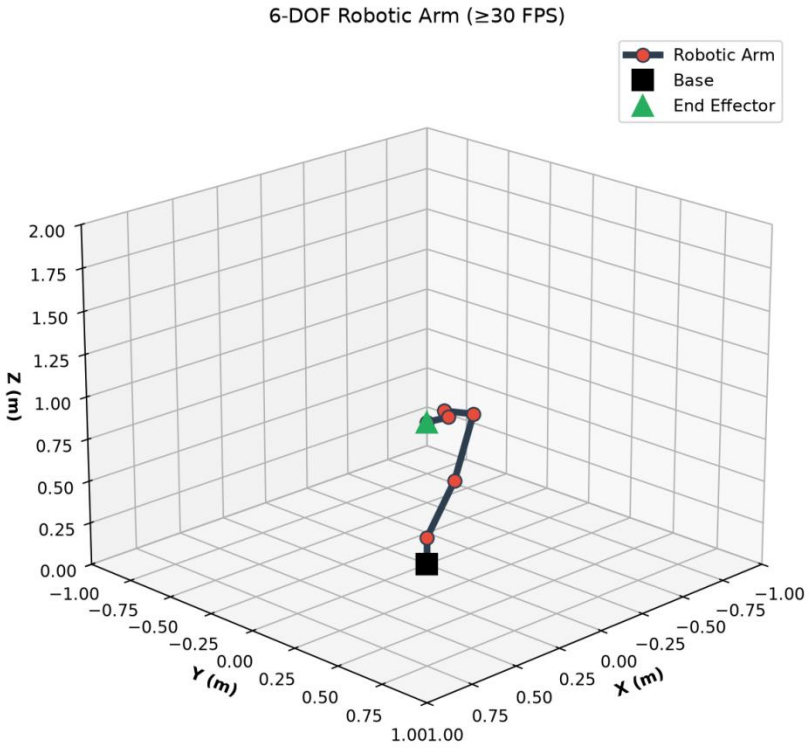


附录 5：三维螺旋与星形轨迹跟踪演示（trajectory.py）

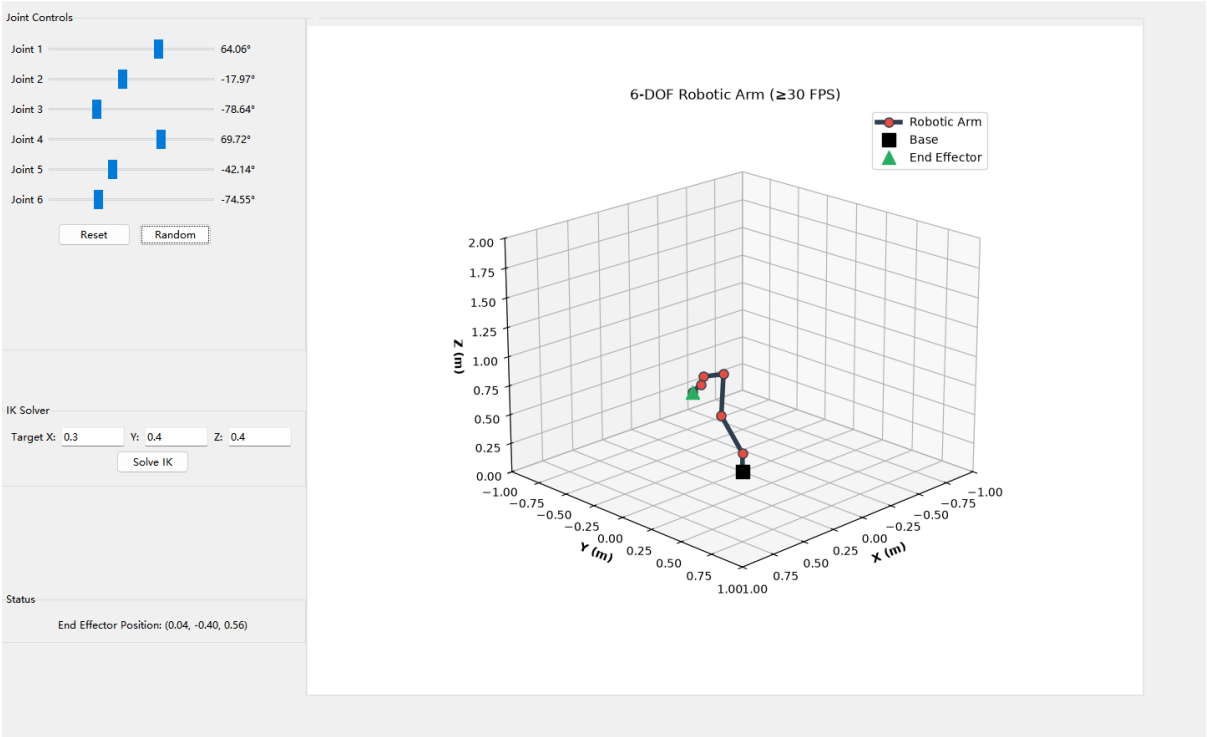
```
==================================
要求 2: 计算效率对比
==================================
方法 [linear ]: 平均耗时 0.3831 毫秒
方法 [cubic  ]: 平均耗时 0.6582 毫秒
方法 [quintic]: 平均耗时 1.4611 毫秒

正在生成 3 的曲线图，请查看弹出的窗口...
正在规划 3D 螺旋线 (可能需要几秒钟解算 IK)...
正在规划五角星平滑轨迹...
有效路径点数: 6 / 6
```

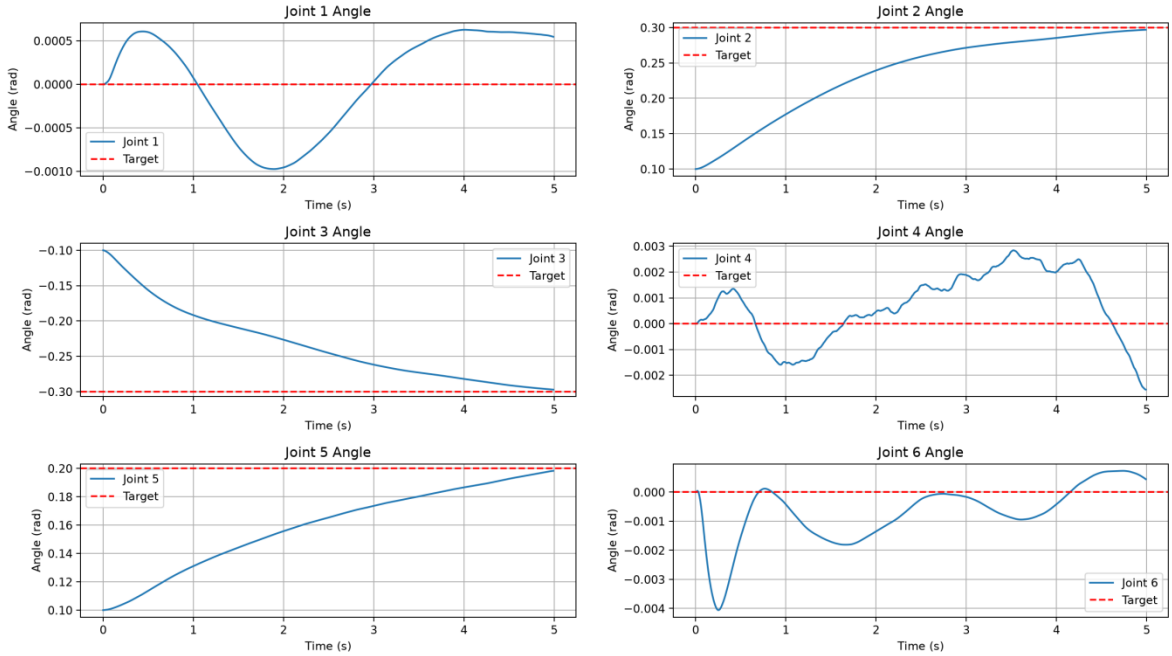
附录 6：轨迹规划模块自测终端输出（trajectory.py）



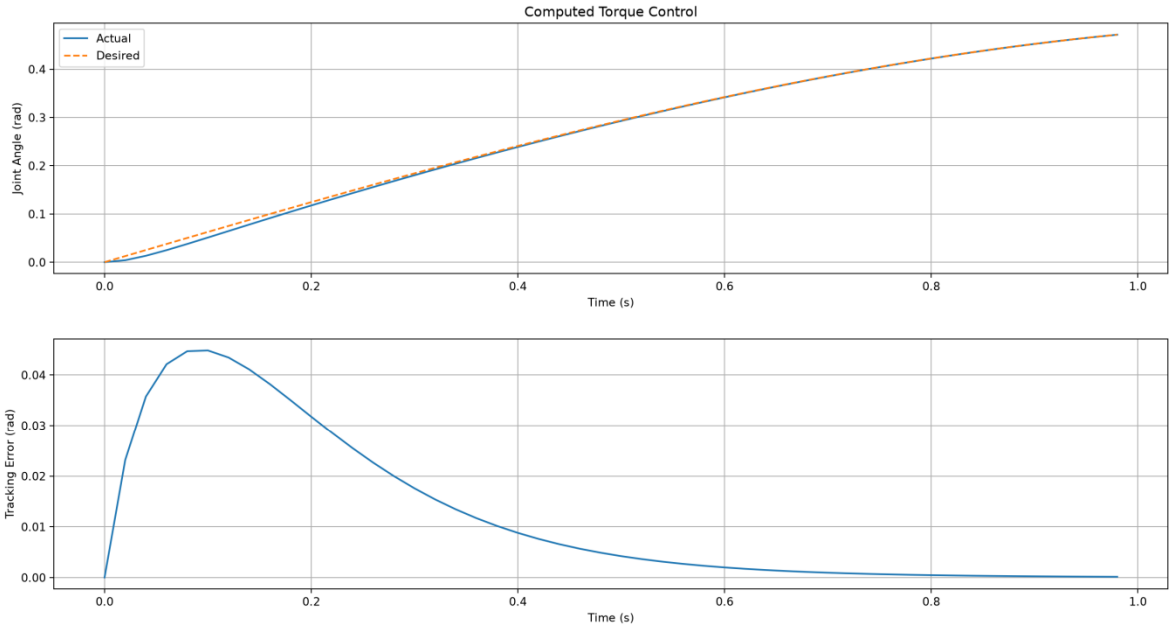
附录 7：三维可视化模块渲染效果（visualizer.py）



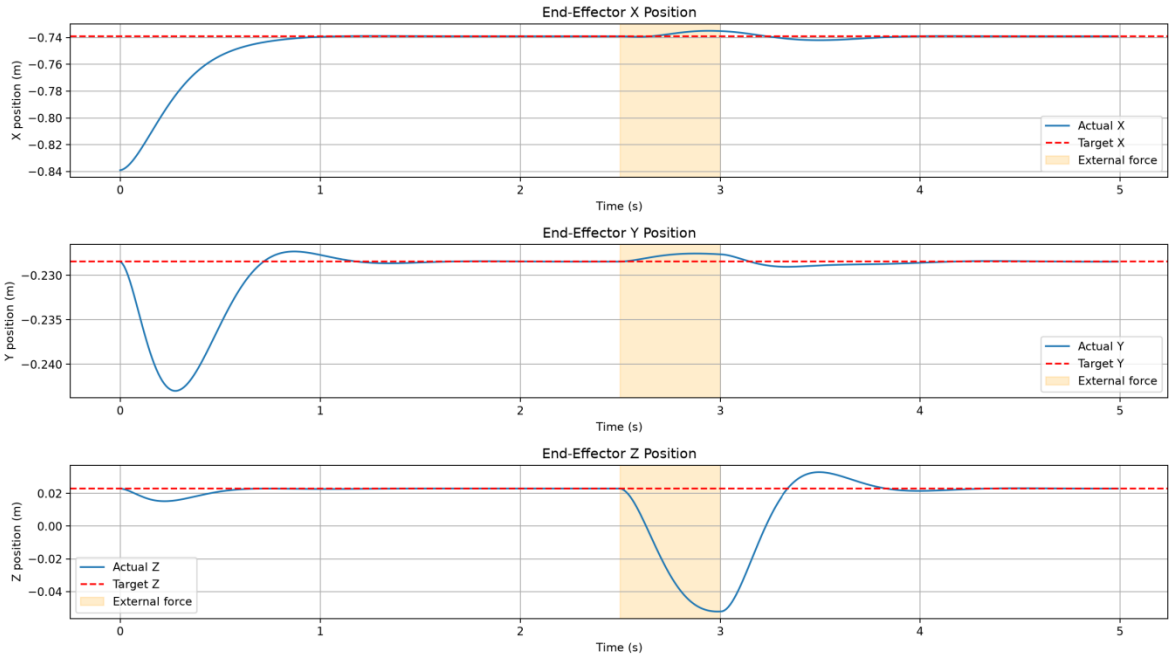
附录 8：GUI 关节交互控制界面（gui_control.py）



附录 9：关节位置保持控制测试（robotic_arm_dynamics.py）



附录 10：计算力矩控制追踪与误差曲线（robotic_arm_dynamics.py）



附录 11：笛卡尔阻抗控制末端位置响应（robotic_arm_dynamics.py）

```
========== 验证质量矩阵 ==========
质量矩阵对角线: [3.8204 3.7344 0.9422 0.1516 0.1312 0.06  ]
条件数: 74.21751486312778

========== 验证重力向量符号 ==========
重力向量 G = [ 0.      30.4683  9.8316 -1.7309  0.5182  0.    ]
Joint 2 (抬起 45°) 的重力矩 G[1] = 30.4683
期望: G[1] > 0（需要正力矩抵抗重力下压）
结果: ✓ 符号正确

========== 运行自由落体测试 ==========
测试1: 重力补偿
重力力矩: [ 0.         36.98853307 16.35179847  0.         0.         0.        ]

测试2: 自由落体仿真
动力学仿真计算中... 虚拟时间推进到了 t = 0.490 秒
仿真时间: 100个时间步
最终角度: [-0.22041726 -2.03794234 -0.38832404 -1.44479493  0.59512275  0.04565529]

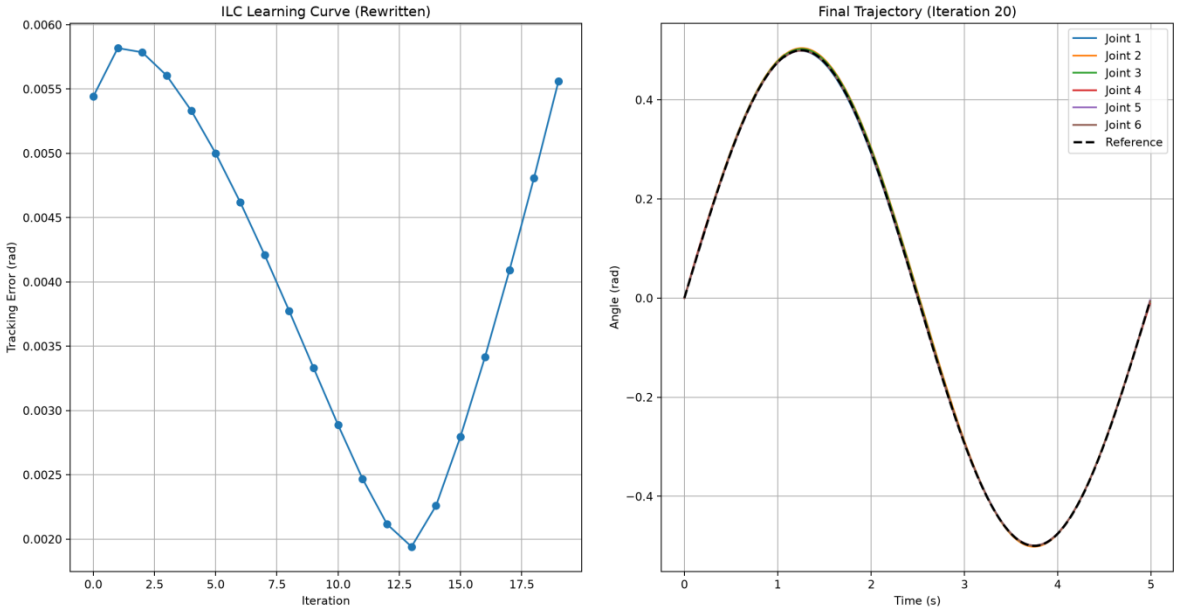
========== 运行重力补偿+PID测试 ==========
重力补偿+PID(带卡尔曼滤波)闭环仿真计算中...
动力学仿真计算中... 虚拟时间推进到了 t = 0.490 秒
动力学仿真计算中... 虚拟时间推进到了 t = 0.990 秒
动力学仿真计算中... 虚拟时间推进到了 t = 1.490 秒
动力学仿真计算中... 虚拟时间推进到了 t = 1.990 秒
动力学仿真计算中... 虚拟时间推进到了 t = 2.490 秒
动力学仿真计算中... 虚拟时间推进到了 t = 2.990 秒
动力学仿真计算中... 虚拟时间推进到了 t = 3.490 秒
动力学仿真计算中... 虚拟时间推进到了 t = 3.990 秒
动力学仿真计算中... 虚拟时间推进到了 t = 4.490 秒
最终误差: [0.0005459  0.00312392 0.00288961 0.00255599 0.00175284 0.00044485]
最大误差: 0.003124 rad
```

附录 12：动力学模型数值验证——惯性矩阵与重力项（robotic_arm_dynamics.py）

```
========== 运行计算力矩控制器测试 ==========
测试计算力矩控制器...
动力学仿真计算中... 虚拟时间推进到了 t = 0.049 秒
动力学仿真计算中... 虚拟时间推进到了 t = 0.199 秒
动力学仿真计算中... 虚拟时间推进到了 t = 0.346 秒
动力学仿真计算中... 虚拟时间推进到了 t = 0.469 秒
动力学仿真计算中... 虚拟时间推进到了 t = 0.619 秒
动力学仿真计算中... 虚拟时间推进到了 t = 0.766 秒
动力学仿真计算中... 虚拟时间推进到了 t = 0.906 秒
计算力矩控制 - 平均误差: 0.0122 rad
计算力矩控制 - 最大误差: 0.0448 rad

========== 运行笛卡尔阻抗控制测试 ==========
初始末端位置: [-0.8391 -0.2285  0.023 ]
目标末端位置: [-0.7391 -0.2285  0.023 ]
阻抗控制仿真计算中...
动力学仿真计算中... 虚拟时间推进到了 t = 0.490 秒
动力学仿真计算中... 虚拟时间推进到了 t = 0.990 秒
动力学仿真计算中... 虚拟时间推进到了 t = 1.490 秒
动力学仿真计算中... 虚拟时间推进到了 t = 1.990 秒
动力学仿真计算中... 虚拟时间推进到了 t = 2.490 秒
动力学仿真计算中... 虚拟时间推进到了 t = 2.990 秒
动力学仿真计算中... 虚拟时间推进到了 t = 3.490 秒
动力学仿真计算中... 虚拟时间推进到了 t = 3.990 秒
动力学仿真计算中... 虚拟时间推进到了 t = 4.490 秒
最终末端位置: [-0.7391 -0.2285  0.0229]
位置误差: 0.0000 m
```

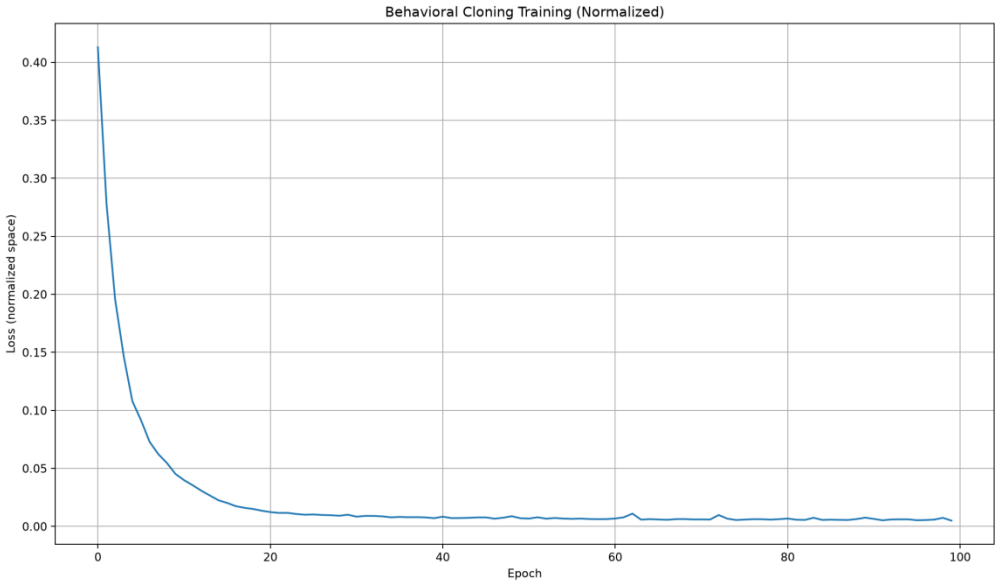
附录 13：前向动力学仿真推进输出（robotic_arm_dynamics.py）



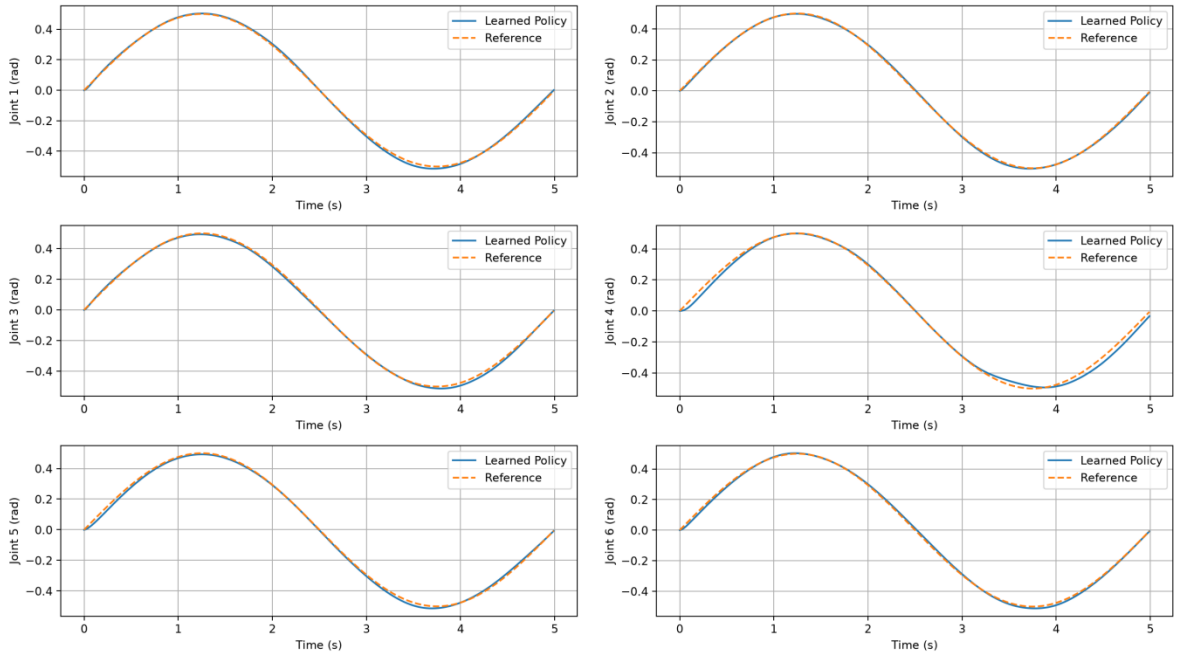
附录 14：ILC 学习曲线与最终轨迹（20 轮迭代）（robotic_arm_optimization.py）

```
开始 ILC 学习（前馈学习 + PD 反馈）...
迭代 1: 误差 = 0.0054 rad
迭代 2: 误差 = 0.0058 rad
迭代 3: 误差 = 0.0058 rad
迭代 4: 误差 = 0.0056 rad
迭代 5: 误差 = 0.0053 rad
迭代 6: 误差 = 0.0050 rad
迭代 7: 误差 = 0.0046 rad
迭代 8: 误差 = 0.0042 rad
迭代 9: 误差 = 0.0038 rad
迭代 10: 误差 = 0.0033 rad
迭代 11: 误差 = 0.0029 rad
迭代 12: 误差 = 0.0025 rad
迭代 13: 误差 = 0.0021 rad
迭代 14: 误差 = 0.0019 rad
迭代 15: 误差 = 0.0023 rad
迭代 16: 误差 = 0.0028 rad
迭代 17: 误差 = 0.0034 rad
迭代 18: 误差 = 0.0041 rad
迭代 19: 误差 = 0.0048 rad
迭代 20: 误差 = 0.0056 rad
```

附录 15：ILC 迭代误差逐轮输出（robotic_arm_optimization.py）



附录 16：行为克隆训练损失曲线（robotic_arm_imitation.py）



附录 17：行为克隆闭环六关节追踪（robotic_arm_imitation.py）

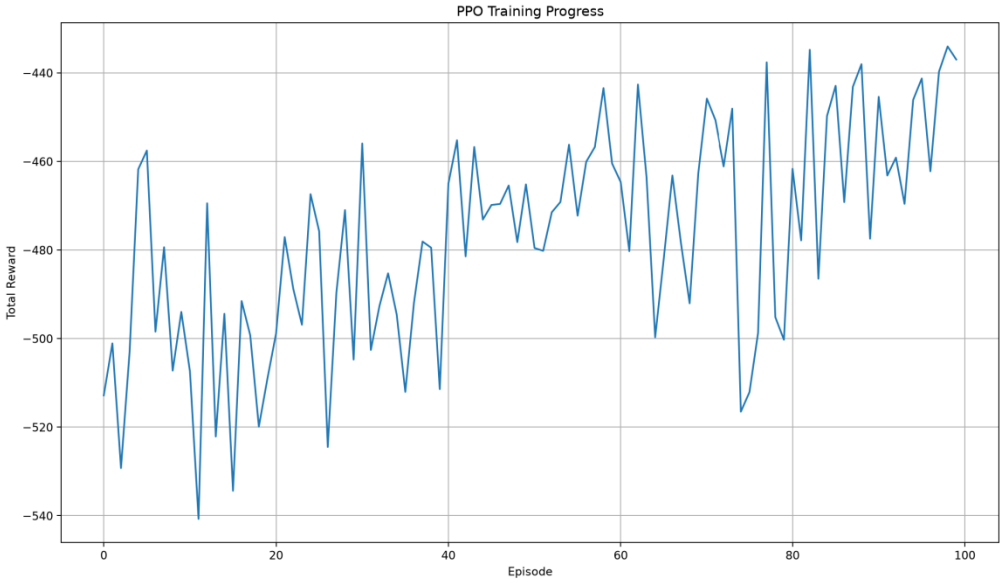
```
收集 20 条专家演示...
完成 2/20 条轨迹
完成 4/20 条轨迹
完成 6/20 条轨迹
完成 8/20 条轨迹
完成 10/20 条轨迹
完成 12/20 条轨迹
完成 14/20 条轨迹
完成 16/20 条轨迹
完成 18/20 条轨迹
完成 20/20 条轨迹

数据集大小: 10000 个样本
状态维度: 24
动作维度: 6
力矩量级（std）: [11.27 13.44  7.32  2.1   0.77  0.31]
开始训练...
Epoch 10/100, Loss: 0.0452
Epoch 20/100, Loss: 0.0135
Epoch 30/100, Loss: 0.0100
Epoch 40/100, Loss: 0.0070
Epoch 50/100, Loss: 0.0070
Epoch 60/100, Loss: 0.0063
Epoch 70/100, Loss: 0.0060
Epoch 80/100, Loss: 0.0062
Epoch 90/100, Loss: 0.0075
Epoch 100/100, Loss: 0.0049

测试学习到的策略...
闭环跟踪 RMSE: 0.0111 rad

模型已保存到 bc_policy.pth
```

附录 18：行为克隆数据采集与训练终端输出（robotic_arm_imitation.py）



附录 19：PPO 训练奖励曲线（robotic_arm_reinforcement.py）

```
开始PPO训练...
Episode 10: Avg Reward = -494.44
Episode 20: Avg Reward = -508.81
Episode 30: Avg Reward = -489.46
Episode 40: Avg Reward = -490.40
Episode 50: Avg Reward = -467.98
Episode 60: Avg Reward = -464.96
Episode 70: Avg Reward = -472.93
Episode 80: Avg Reward = -476.61
Episode 90: Avg Reward = -458.13
Episode 100: Avg Reward = -449.76
```

附录 20：PPO 训练回合奖励终端输出（robotic_arm_reinforcement.py）